

Chương này trình bày chi tiết những đóng góp nổi bật nhất, bao gồm các kiến trúc, giải thuật và mô hình thiết kế được tác giả tự xây dựng từ đầu (from scratch) nhằm giải quyết các bài toán khắt khe trong hệ thống Trợ lý ảo Giáo dục. Thay vì phụ thuộc vào các nền tảng đóng gói sẵn của bên thứ ba, tôi đã phân tích và thiết kế lại hàng loạt các cụm tính toán nhằm giải quyết các điểm nghẽn hiệu năng, hạn chế sự ảo giác thông tin và tối ưu hóa luồng thực thi trong môi trường đòi hỏi độ chính xác tuyệt đối như giáo dục học thuật.

0.1 Tự xây dựng bộ máy truy xuất kết hợp (BM25 + Semantic + RRF) từ mức cơ sở

0.1.1 Dẫn dắt vấn đề

Trong hệ thống RAG, để mô hình có thể bám sát nội dung sách giáo khoa, hệ thống cần khả năng truy xuất đồng thời theo từ khóa chuyên ngành chính xác (Keyword-based) và theo ngữ nghĩa tương đồng (Semantic-based). Giải pháp biểu diễn từ khóa truyền thống như TF-IDF thường gặp hạn chế nghiêm trọng do thiên lệch độ dài văn bản (text length bias), khiến các phần nội dung dài dễ dàng lấn át các định nghĩa ngắn gọn - vốn rất phổ biến trong sách giáo khoa - dù mức độ liên quan của chúng không cao.

Ở một khía cạnh khác, việc ứng dụng ngay các bộ máy tìm kiếm công nghiệp công kênh như Elasticsearch hoặc thư viện nền tảng như FAISS lại đặt ra sự hao phí tài nguyên không cần thiết. Tập dữ liệu cấu trúc sách giáo khoa của dự án chỉ dừng lại ở phạm vi gần 2348 khối văn bản phân mảnh tĩnh. Hơn nữa, những thư viện truy xuất vector cục bộ như FAISS không hỗ trợ sẵn (native support) thuật toán BM25 gốc, gây ra nhiều khó khăn trong việc hợp nhất.

0.1.2 Giải pháp đề xuất

Thay vì phụ thuộc thư viện, tôi đã tự lập trình lại thuật toán BM25 và khung tìm kiếm ngữ nghĩa độc lập. Thuật toán BM25 được cấy nội bộ với hai tham số kiểm soát $k_1 = 1.2$ và $b = 0.75$. Tham số k_1 không chế mức độ bão hòa sự xuất hiện của từ khóa, trong khi b điều phối sự tinh duyệt, vừa đủ để phạt bớt các văn bản quá dài mà không tàn phá các đoạn định lý học thuật quan trọng cấu trúc nguyên vẹn.

Về tìm kiếm ngữ nghĩa, luồng thực thi sử dụng mô hình nhúng `dangvantuan/vietnamese-document-embedding` hỗ trợ vectơ 768 chiều tối ưu hóa trực tiếp cho tiếng Việt. Các vectơ biểu diễn văn bản (embeddings) được trích xuất hoàn toàn khả lập trước (pre-computed). Tôi áp dụng thêm phép chuẩn hóa L2 lên hệ vectơ, giúp việc tính toán độ đo khoảng cách (Cosine Similarity) cho không gian người dùng tra cứu chỉ quy lại thành những phép nhân ma trận dấu chấm vô cùng giản đơn.

Khâu kết hợp 2 luồng điểm số truy cập này lại với nhau được định nghĩa bằng kỹ thuật Dung hợp Hạng nghịch đảo (Reciprocal Rank Fusion - RRF). Công thức RRF được tùy biến cấu hình tại lớp code với hằng số $k = 60$. Hằng số mượt này đã được tôi phân tích cẩn thận, nó khóa chặn yếu điểm đột biến bảng xếp hạng (rank bias), tạo ra sự hòa quyện tuyệt đối giữa hai dạng truy xuất nhằm đảm bảo không một kỹ thuật nào được phép chiếm ưu thế hoàn toàn so với kỹ thuật còn lại.

0.1.3 Kết quả đạt được

Sự đánh đổi công sức lập trình này tạo ra kết quả vô cùng tuyệt vời. Mã nguồn vận hành siêu nhẹ (lightweight), hoàn toàn tương thích và dễ gỡ lỗi do mọi quá trình sinh điểm thứ hạng đều tường minh. Hệ thống tự triệt tiêu đi các độ trễ gọi dịch vụ cơ sở dữ liệu bên ngoài, trong khi vẫn đạt được tính thống nhất cả về độ chính xác của từ khóa đặc thù chuyên ngành lẫn mức độ thông hiểu ý nghĩa ngữ cảnh học bài.

0.2 Thiết kế Điều phối viên mức mã nguồn (Code-level Orchestrator)

0.2.1 Dẫn dắt vấn đề

Khuynh hướng hiện đại khi xây dựng chuỗi thao tác LLM thường lạm dụng các khung framework có sẵn như LangChain, LlamaIndex hay LangGraph. Sự mượt mà tiện dụng của chúng đi kèm theo vô vàn đoạn mô đun ẩn (opaque flow/magic abstractions) làm cho dấu vết dịch chuyển chuỗi dữ liệu rất mờ昧. Khi xảy ra sai sót dữ liệu do lệnh prompt sinh ra từ thư viện mâu thuẫn hệ thống, chuỗi khối này gây trở lại rào cản chết (heavy dependency), làm việc gỡ lỗi trở thành thảm họa.

0.2.2 Giải pháp đề xuất

Thay vì đóng hộp đồ án, tôi đã cấu trúc và tự thiết kế một *Orchestrator* (Điều phối viên tác vụ) hoàn toàn tại lớp mức lệnh cơ sở gốc (pure-code nodes). Orchestrator được định hình là "Thin controller" (Bộ xử lý màng xấp xỉ), nó điều phối dòng xử lý xoay quanh bốn hàm tự nhiên: `ContextAnalyzer`, `SessionManager`, `ActionPlanner` và `RAG Agent` mà không cần nhúng LangChain.

Ý tưởng chủ đạo ở đây là dồn tất cả tiến trình gọi LLM đắt đỏ (LLM call) về mức tính tối giản: Hệ thống bắt buộc dùng LLM Call duy nhất tại lúc nhận diện (Intent Detection). Tất cả các luồng trung chuyển dữ liệu ở tầng `ActionPlanner`, việc nối context từ bộ đệm... đều được tính toán bằng Python nguyên thủy cùng các quy luật cây trạng thái. Không có mô hình ngôn ngữ nào tham gia cản trở ở bước điều hướng, đảm bảo luồng đi luôn an toàn, không có ảo giác mã rẽ.

0.2.3 Kết quả đạt được

Mô hình Orchestrator này giúp toàn bộ kết cấu độ trễ (latency) của toàn chuỗi dịch vụ bị triệt tiêu, trả lại tốc độ phản hồi gần mức API nguyên chất thuần túy. Quá trình theo vết vòng đời câu hỏi thông qua Console Logging (nhằm quan sát tham số mỗi nhử hay số block ngữ cảnh nạp vào) cực kỳ dễ dàng. Tuy sự thay thế framework công nghiệp đồng nghĩa với việc đòi hỏi sinh viên phải chịu sức ép mở rộng hệ đa tác vụ thủ công trong tương lai, sự bám rễ linh hoạt này hoàn toàn vô cấp khóa nền tảng (zero vendor lock-in).

0.3 Kiến trúc Adaptive RAG Agent với suy luận chọn lọc Heuristics

0.3.1 Dẫn dắt vấn đề

Một quy trình xử lý khối dữ liệu (RAG pipeline) chung không thể bao phủ nổi các phong cách tương tác biến hóa. Sẽ là lãng phí nếu đem quy trình gom tập RAG đi lọc toàn bộ một tệp thư viện lớn khi đối phương chỉ hỏi cách truy cập cấu trúc phân tập (Curriculum) theo sách. Nhưng nếu lạm dụng LLM vào vai trò phán đoán "Tôi nên truy xuất kiểu gì vào lúc này", hệ thống một lần nữa sẽ tắc nghẽn về tốc độ và chi phí hoạt động (cost latency).

0.3.2 Giải pháp đề xuất

Tôi đã xây dựng thành phần *Adaptive RAG Agent* hỗ trợ cho luồng đi nội tại và không giao nhiệm vụ quyết định cho LLM, thay vào đó là việc khai thác khả năng của thuật toán dựa trên suy nghiệm (Heuristics-based `QueryClassifier`).

Bộ phân loại quy tắc truy xuất dựa trên khớp từ vựng chuẩn (keyword matching) phân tích chủ đề định nghĩa (topic hints) từ lịch sử chat, xác định bài toán trong mức dưới 1/100 giây thông qua các tập Regular Expression để dẫn dắt đến một trong bốn chiến lược sau:

- **STANDARD (Chuẩn):** Tập kết hoàn chỉnh luồng BM25 kết hợp Semantic Fusion RRF và sau đó là Reranker phục vụ khai phá câu hỏi học sâu đa hướng.
- **BROAD (Rộng):** Áp dụng luồng xử lý thu mục tiêu tổng quát (metadata lookup for objective chunks), bỏ hẳn quá trình Xếp hạng lại (Reranking). Cửa ngõ này đáp ứng các câu tổng hợp mô tả mà giúp tốc độ truy vấn tăng xấp xỉ 10 lần.
- **CURRICULUM (Mục lục):** Thu gọn phạm vi tra cứu trên tập tin đồ hệ thống sách tạo dàn ý.
- **HRAG (Hierarchical):** Mở bối cảnh theo hai cấp (Thô và Tinh), phục vụ riêng cho các phân đoạn định nghĩa phức tạp đa tính liên đới bài tập chéo.

Để giải quyết tính cố định yếu điểm của quy luật, một tập thuật điều phối lùi (fallback logic) đóng vai trò lá chắn. Ví dụ, nếu luồng BROAD lọc dữ liệu không xuất trả ra đủ số lượng mảnh ngữ nghĩa (dưới 3 chunks), hệ thống tự kích hoạt quay ngược luồng tiêu chuẩn STANDARD nhằm bảo vệ an toàn bối cảnh trước khi chuyển đi.

0.3.3 Kết quả đạt được

Áp dụng cơ chế Heuristic giảm tải quá tải hệ thống khi phải thực hiện hàng chục mô hình truy thu, nhưng vẫn giúp LLM sinh đáp án với nguồn cấp dữ liệu (grounding) chuẩn nhất.

0.4 Nhận diện Ý định Hai Cấp (2-Level Intent Detection) giải tỏa độ trễ

0.4.1 Dẫn dắt vấn đề

Lớp tiếp điểm nhận giao tiếp tự nhiên đòi hỏi việc xác định luồng tác vụ. Nếu sử dụng LLM tạo kịch bản xử lý tự lập hoàn toàn theo dạng Agent pattern vòng lặp (gọi LLM liên tiếp tạo ý định, gọi tiếp LLM phân tách bài, tiếp tục gọi LLM lập action plan), một chu trình dính kèm tốn kém tối thiểu 3 đến 5 API requests dẫn đến hàng chục giây độ trễ rập khuôn.

0.4.2 Giải pháp đề xuất

Thiết kế đột phá được cấy vào hệ thống bằng chiến lược **Nhận diện ý định Hai cấp (2-Level Intent Detection)**. Tại Cấp độ 1, kỹ thuật đẩy một lời gọi LLM duy nhất với mức kiểm soát tham số sáng tạo ngắt chặt ở mức nhiệt *temperature* = 0.1. Chỉ dẫn (instruction) ép buộc hệ thống bóc tách hoàn chỉnh các siêu thông tin chứa: Tập nhóm Lệnh (intent), Loại nhiệm vụ (task type), Khoảng nội dung (topic), Lớp học và Đầu sách. Nội dung JSON trả về ép nhận diện song song cho nhiều ý định gõ trong cùng một thời điểm, tuy nhiên tôi chốt giới hạn ngưỡng (Max 3 Intents) để chặn đứt mọi hậu quả suy diễn lây lan mạng (state explosion).

Tại Cấp độ 2, Module *ActionPlanner* khởi động với luồng dữ liệu truyền vào là mã tính không chế, chuyển chuỗi dịch tham chiếu từ JSON List sang mã lệnh vận hành Enum nguyên khối thông thường.

0.4.3 Kết quả đạt được

Bằng cấu trúc này, đồ án đã tách ghép sự tự trị quá đáng của hệ mạng đa tác nhân. Phương châm thiết kế “1 lượt LLM bóc tách gốc rễ kết hợp 0 lượt LLM vận hành lệnh phân phối” định hình trơn tru hiệu suất, cắt đứt áp lực độ trễ đứt gãy mà thông số độ ổn định hệ thống vẫn cực kì tuyệt đối.

0.5 Hệ thống Phản biện khép kín (Self-Reflection bằng QuestionValidator)

0.5.1 Dẫn dắt vấn đề

LLM dễ bị tấn công suy diễn bất biến (hallucination) ở bất kể phiên bản năng lực mạnh thế nào, đặc biệt khi được giao những cấu trúc tạo khuôn ra bộ kiểm tra trắc nghiệm (Quiz JSON Arrays) hay bố cục trình diễn. Sự lọt lưới một câu trả lời sai đi ngược quan điểm trong sách chuẩn sẽ biến hệ thống giáo dục thành mối nguy hiểm thảm họa.

0.5.2 Giải pháp đề xuất

Nền tảng sinh mã đã được tách ly làm hai để triển khai **Hệ thống Phản biện Tự động hai LLM (Dual-LLM Cross-check)**. Toàn bộ kết quả mảng câu hỏi trắc nghiệm sinh ra từ LLM thực thi sẽ không được xả thẳng lên mạng lưới trình duyệt, mà lập tức bị bắt lại tại Cổng kiểm duyệt `QuestionValidator`. Tác nhân ở cổng này không làm nhiệm vụ sinh đáp án, mà mang riêng System Prompt đóng vai giám thị, đối sánh cấu trúc nội dung từ khóa được tạo ra với các luật khắt khe trong Sách Giáo Khoa (Context Source) và Khung tiêu chí xếp dạng câu (Rubrics).

Nếu kết quả đối sánh đánh điểm không đạt cấu trúc, luồng sinh cấu trúc sẽ nhận một lời báo lỗi hồi quy tự suy sửa chữa (retry generation). Tách biệt vòng xoáy thời gian, quá trình này bị hệ thống gài lặp tối đa hai lượt.

0.5.3 Kết quả đạt được

Chỉ những nội dung mang phân định "Approved" lọt qua khe mới được gửi về giao diện. Tính hệ thống định hướng cấu trúc (Structured output limit) biến mọi phiên sinh của đề án thành tệp dữ liệu không góc lồi, nâng cấp tính ứng dụng của nó đến cấp hoạt động ổn định ở tính thương mại thực tế do dữ liệu trả về sở hữu tính căn bản an toàn.

0.6 Chiến lược Quản lý Trạng thái Bộ nhớ linh hoạt (Memory State Strategy)

0.6.1 Dẫn dắt vấn đề

Phiên bản Trợ lý ảo duy trì một phiên đàm thoại hỏi bài dài hàng giờ sẽ tích nập chuỗi tin nhắn rất lớn. Việc gom tổng lịch sử thô (full text history) vào luồng prompt mới để giúp LLM bắt mạch hội thoại vừa lãng phí Token chi phí quá đắt đỏ lại dễ làm loãng dữ kiện cốt lõi ở câu lệnh đầu do hiện trạng giới hạn sổ nhớ mô hình chặn đầu cản đuôi. Một giải pháp sử dụng cơ sở dữ liệu lớn để quản lý lại đánh mất tiêu chí gọn nhẹ linh hoạt.

0.6.2 Giải pháp đề xuất

Tôi đã áp dụng giải pháp quản trị *Windows-Context* (Khoanh vùng giới hạn) tích hợp trạng thái *Session* lưu vết *JSON* cục bộ. Cơ chế đếm cửa sổ chặn giới hạn

nap `max_messages=10` (chiết suất trượt đọc lưu 10 lượt giao tác mới nhất) điều hướng dữ liệu. Mạch liên kết không bị tan rã, token nội dung không phình to nhờ nhét dẫn đến các luồng rác. Tuy giải quyết luồng trượt nhưng bảo toàn bền vững phiên, mỗi người dùng gõ câu lệnh đều lưu tệp sao lưu phiên dạng file định danh tính đĩa cứng riêng.

Bảng mạch đặc biệt dành riêng cho tính năng báo sửa luyện thi rẽ nhánh bằng khối `QuizRound`. Phân đệm tách rời tập ghi chú dữ liệu lịch sử hội thoại chuẩn giúp học sinh có thể yêu cầu Trợ lý lục soát lại “Câu sai môn đó lúc nãy” mà nội dung này không hề nhồi đệm xen lẫn vào hệ thống lưu trữ giao tiếp thông thường.

0.6.3 Kết quả đạt được

Chiến lược đem đến thiết kế kiến trúc theo cơ sở vô trạng thái (Stateless server), nơi RAM hệ thống không bị tràn do tiêu hủy trạng thái tự nhiên, bù lấp bằng truy xuất tập tệp chuẩn rất nhẹ cục bộ. Sự tách gỡ này không chỉ tinh giản kiến trúc xử lý vận hành tiết kiệm chi phí mà làm bàn đạp để tinh chỉnh sau này mở rộng quy mô đổ xuống DB lưu trữ dữ liệu tập trung.

0.7 Thiết kế Hệ thống Đa tác nhân (Multi-Agent System) cho quá trình Sinh Slide Tự động

0.7.1 Tổng quan kiến trúc: Mô hình Supervisor (Supervisor Pattern) và Lựa chọn Sub-Agent

Nhu cầu tự động hóa việc tạo bài giảng (Slide) đòi hỏi hệ thống phải xử lý lượng thông tin đồ sộ từ việc trích xuất kiến thức, tóm tắt, tìm kiếm hình ảnh minh họa, cho đến việc sinh câu hỏi củng cố. Nếu gom tất cả các nhiệm vụ này vào một Agent duy nhất (Monolithic Agent), ngữ cảnh truyền vào (context window) từ RAG, kế hoạch (plan) và nội dung sẽ trở nên khổng lồ. Điều này dẫn đến hiện tượng quên lửng giữa chừng (Lost in the middle) khiến mô hình mất tập trung, không định hướng được các nhiệm vụ rời rạc.

Để giải quyết bài toán trên, tôi đã áp dụng mô hình **Supervisor Pattern**. Trong kiến trúc này, một nút chỉ huy (Supervisor) đóng vai trò trung tâm tiếp nhận yêu cầu và định tuyến tuần tự đến các tác nhân con (Sub-Agents) chuyên biệt. Mô hình này mang lại khả năng kiểm soát luồng chặt chẽ và rất thuận tiện để mở rộng hay nâng cấp từng thành phần độc lập. Các Sub-Agent được cân nhắc dựa trên tiến trình sư phạm thực tế của một bài giảng chuẩn mực, phân tách quy trình ra làm 5 Sub-Agent xử lý theo trình tự chu trình làm việc (workflow) dưới đây.

0.7.2 OutlineAgent: Thiết lập Dàn ý Tổng quát

Nhiệm vụ: Tạo dàn ý (outline) tổng quát cho toàn bộ slide bài giảng.

Lý do tách biệt: OutlineAgent chỉ tập trung trả lời câu hỏi "Bài giảng này nên có những slide nào, tiêu đề là gì?". Việc phân tách đầu tiên này cắt giảm được luồng tư duy phức tạp của ngôn ngữ, giúp mô hình trở nên nhẹ hơn rất nhiều, bảo đảm cấu trúc đầu ra tuân thủ đúng mạch logic cốt lõi.

0.7.3 ContentAgent: Phát triển Nội dung Chi tiết (Cơ chế Fan-out)

Nhiệm vụ: Dựa trên bộ dàn ý có sẵn được trích xuất từ OutlineAgent, ContentAgent tiến hành đắp thịt bằng việc sinh nội dung chi tiết cho từng slide.

Cơ chế Fan-out (Phân tán): Các node xử lý được thiết kế độc lập, không phụ thuộc vào chuỗi tuần tự đăng trước hoặc đăng sau. ContentAgent khởi tạo luồng song song đắp thông tin cho từng chunk dữ liệu slide độc lập, giúp tốc độ sinh bài giảng được đẩy nhanh lên tối đa.

0.7.4 MediaAgent: Tích hợp Dữ liệu Trực quan

Nhiệm vụ: Thu thập và gợi ý các hình ảnh, đồ họa dạng GIF hoạt hình minh họa cho bài giảng.

Lý do cần thiết: Bất kỳ một bài giảng sư phạm tốt nào cũng cần có phần khởi động (hook) kích thích thị giác, hoặc chèn các ví dụ hình ảnh sinh động giúp truyền đạt những khái niệm trừu tượng một cách tổng quan và dễ hiểu. Việc này đánh mạnh vào tính định hướng tương tác (engagement) cho người học trong hệ thống giáo dục thay vì những khối văn bản khô khan.

0.7.5 QuizAgent: Sinh bài tập Hệ thống hóa Kiến thức

Nhiệm vụ: Tự động sinh ra bộ câu hỏi luyện tập trực tiếp đặt ở cuối bài giảng.

Lý do cần thiết: QuizAgent được tái sử dụng hoàn toàn (reuse code) từ dịch vụ QuizService cốt lõi đã có trước đó. Trong các bài giảng giáo dục, luôn cần tồn tại phần củng cố kiến thức lúc chốt bài. QuizAgent tự động chèn phần ôn tập cuối giờ này, giúp gia tăng giá trị trọn gói của một tiết học tự động.

0.7.6 MergeAgent: Tổng hợp và Kiểm duyệt (Bắt buộc)

Nhiệm vụ: Tổng hợp lại kết quả chấp vá từ các luồng Outline, Content, Media và Quiz thành một khối bản trình chiếu duy nhất và hoàn chỉnh.

Cơ chế Fan-in (Hội tụ): Nút tác nhân này thiết lập cơ chế vòng chặn đợi tất cả các Sub-Agent trước đó xử lý xong, đóng vai trò như một Cổng bảo chứng chất lượng (Quality Gate) cuối cùng. Nó bắt buộc kiểm tra độ hoàn chỉnh của dữ liệu JSON chấp vá để chắc chắn không một slide hay hình ảnh, câu trắc nghiệm nào bị bỏ sót hay gây luồng trước khi gửi kết quả về cho hệ thống hiển thị phía người dùng.